

Investigating System Requirements

Why software projects fail before a single line of code is written,
and how analysts make sure this one doesn't.

The airport that couldn't handle a suitcase

- **1990s:** Denver builds a brand-new international airport. Its showpiece: a fully automated baggage system, computer-controlled carts on miles of track, thousands of motors and sensors, routing any bag to any gate.
- **The promise:** the most efficient airport in America. The software contract: ≈ \$200M.
- **The reality:** in testing, bags were crushed, jammed, and routed to the wrong places. The system could not keep up with real-world conditions.
- **The cost:** the airport opened 16 months late. Delay costs ≈ \$560M. A temporary manual system had to be built.
- **The ending:** the automated system was never fully used and was officially abandoned in 2005.

\$560M

in delay costs

+ 16 months late · system scrapped

ASK YOURSELF

Was this a coding failure, or did it fail earlier than that?

It isn't just airports

HealthCare.gov: USA, 2013

- Launched Oct 1, 2013, as the national health-insurance marketplace, the site immediately buckled; most users could not complete enrollment.
- Government Accountability Office (2014): the program lacked “effective planning or oversight practices.”
- Office of Inspector General (2014): total cost reached \$1.7 billion.

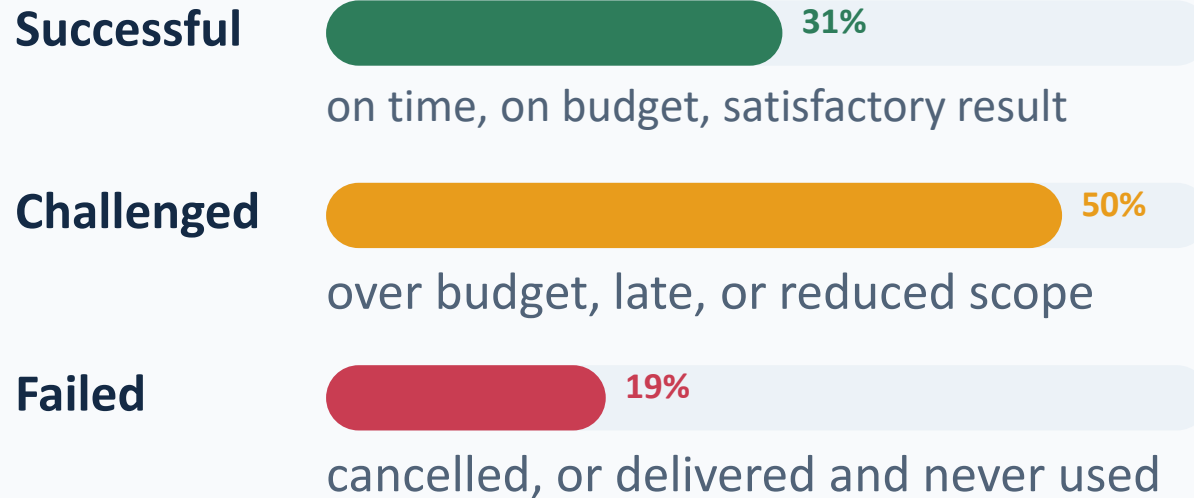
NHS National Programme for IT: UK

- Plan: ~£2.3B over 3 years to digitize health records, called the world's largest civilian IT programme.
- Cancelled in September 2011.
- By 2013 the abandoned effort had cost ~£10B (The Guardian), later estimates ran even higher.

Same question three times: **nobody wrote bad loops. So what actually went wrong?**

Most IT projects don't fully succeed

Standish Group CHAOS Report (2020):



The #1 defense: investigate requirements BEFORE the code, while mistakes are still cheap to fix.

THE COMMON THREAD

In post-mortems of failed projects, problems with requirements are consistently among the top root causes:

- Missing or incomplete requirements
- Users/stakeholders not involved
- Requirements that kept changing
- Unclear priorities, everything “essential”

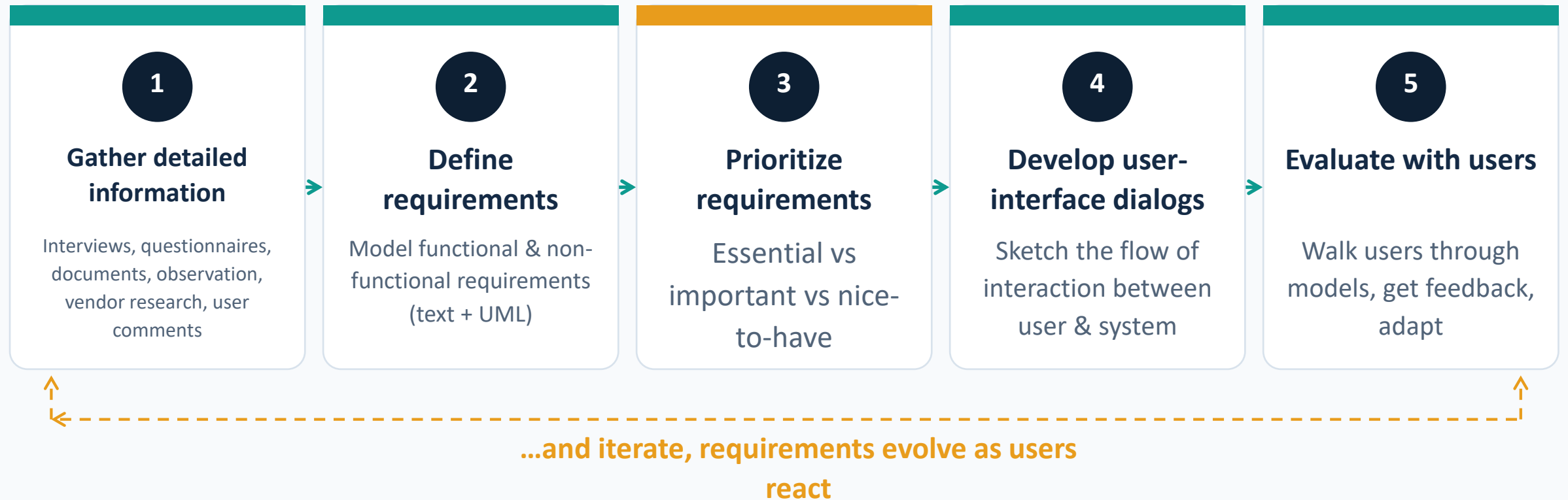
Every one of them is an ANALYSIS problem, which is why this course exists.

Today

By the end of this lecture you will be able to:

- LO1 Classify requirements as functional or non-functional, and organize them with FURPS+.
- LO2 Identify stakeholders of a system: internal / external × operational / executive.
- LO3 Choose and justify information-gathering techniques for a given situation.
- LO4 Read AND draw a UML activity diagram with swimlanes for a business workflow.
- LO5 Trace one requirement from text → code → test (and explain why the ID matters).

What analysts actually do: the requirements loop



The SDLC context: a project starts by identifying the problem, getting approval, and planning, then this loop IS the heart of analysis.

1

What IS a requirement?

Functional vs non-functional · FURPS+ · the anatomy of a good one

System requirements come in two flavors

FUNCTIONAL requirements

The activities the system **must** perform, the business uses, the functions users carry out. Documented as use cases.

Examples:

- ✓ “A customer can search the catalog by keyword.”
- ✓ “A clerk can enter a phone order with up to 15 items.”
- ✓ “The system calculates Mountain Bucks on completed orders.”

NON-FUNCTIONAL requirements

Other characteristics of the system, constraints and performance goals. They shape the architecture.

Examples:

- ✓ “Support 2,000 concurrent shoppers during the holiday sale.”
- ✓ “Catalog searches return results in under 2 seconds.”
- ✓ “The system is available 99.9% of business hours.”

FR = what the system does. NFR = how well it does it.

FURPS+ : the requirements classifier

F

Functionality

What it does:
behaviours, use cases,
business rules.

“Clerk can enter a
phone order.”

U

Usability

Human factors:
learnability, error
messages,
accessibility.

“Checkout in ≤ 6 steps,
no training.”

R

Reliability

Availability, failure
rates, recovery.

“99.9% available in
business hours.”

P

Performance

Speed, capacity,
throughput.

“Search < 2 s at 1,000
users.”

S

Supportability

Testability,
extensibility,
installability,
maintainability.

“New store added
without code change.”

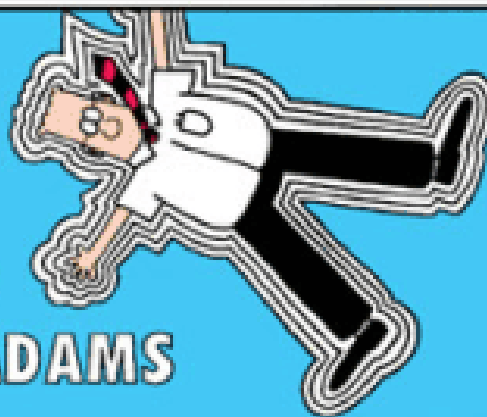
+ plus the constraints: Implementation (reuse SCM's Java/Oracle stack) · Interface (UPS/FedEx shipment APIs) · Operations (runs 24/7 in the existing data center) · Packaging (installable at retail stores) · Legal (tax & consumer law, card-data rules)

Heads-up: the 5th letter is Supportability: not Security. Security requirements are real and vital; teams typically file them under Supportability/Reliability or add an explicit category. FURPS+ comes from Grady Booch at Hewlett-Packard, popularized at Rational.

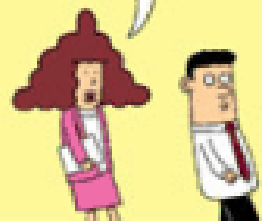


DILBERT®

BY
SCOTT ADAMS



I'LL NEED TO KNOW
YOUR REQUIREMENTS
BEFORE I START TO
DESIGN THE SOFTWARE.



E-mail: SCOTTADAMS@AOL.COM

FIRST OF ALL,
WHAT ARE YOU
TRYING TO
ACCOMPLISH?



I'M TRYING TO
MAKE YOU DESIGN
MY SOFTWARE.



© 2004 Scott Adams, Inc./Dist. by UFS, Inc.

I MEAN WHAT ARE
YOU TRYING TO
ACCOMPLISH WITH
THE SOFTWARE?



I WON'T KNOW WHAT
I CAN ACCOMPLISH
UNTIL YOU TELL ME
WHAT THE SOFTWARE
CAN DO.



UFS-04

TRY TO GET THIS
CONCEPT THROUGH YOUR
THICK SKULL: THE
SOFTWARE CAN DO
WHATEVER I DESIGN
IT TO DO!



www.dilbert.com

CAN YOU DESIGN
IT TO TELL YOU
MY REQUIREMENTS?



Quick fire: F or NFR: and which FURPS letter?

1

“A customer can track a shipment by order number.”

F / NFR-__ · FURPS letter: __

2

“The system must be available 99.9% of business hours.”

F / NFR-__ · FURPS letter: __

3

“A first-time user can complete checkout without any training.”

F / NFR-__ · FURPS letter: __

4

“95% of catalog searches must return within 2 seconds.”

F / NFR-__ · FURPS letter: __

5

“Credit-card numbers must be encrypted when stored.”

F / NFR-__ · FURPS letter: __

6

“The system prints a packing slip for every warehouse order.”

F / NFR-__ · FURPS letter: __

Trap alert: #5 is the one everyone argues about. Be ready to defend your answer.

Anatomy of a good requirement

✓ Unambiguous

exactly one reading

✓ Verifiable

a test can prove it

✓ Atomic

one thing per statement

✓ Consistent

no contradictions

✓ Traceable

has an ID → code → test

✓ Feasible

Possible in time & budget

✗ BEFORE

“The system shall be fast, secure and user-friendly.”

- Three requirements crammed into one (not atomic)
- Nothing measurable: fast to whom? secure against what?
- Untestable → unprovable → guaranteed dispute at delivery

✓ AFTER (3 requirements, 3 IDs)

- PERF-01: “95% of searches return within 2 s at 1,000 users.”
- SEC-01: “Card data encrypted at rest; no card data in logs.”
- USE-01: “First-time user completes checkout in ≤ 6 steps.”

2

Who do we ask?

Stakeholders: the people requirements actually live in

Stakeholders: persons with an interest in the system's success

INTERNAL × OPERATIONAL

Work with the system daily, inside the org

- Phone/mail order clerks
- Warehouse & shipping staff
- Marketing staff maintaining the online catalog

EXTERNAL × OPERATIONAL

Touch the system daily, outside the org

- Customers (web, phone, retail)
- External shippers: e.g., UPS, FedEx
- Retail store cashiers (partner stores)

INTERNAL × EXECUTIVE

Don't operate it; consume its info or fund it

- Marketing, sales, accounting & finance managers
- Senior executives
- IT management (own the architecture)

EXTERNAL × EXECUTIVE

Outside interest in outcomes

- Investors / owners
- Partners in co-marketing deals
- Regulators & tax authorities (e.g.)

RMO's stakeholder list: what's missing?

FROM THE PROJECT CHARTER

- ✓ Phone/mail sales order clerks
- ✓ Warehouse & shipping personnel
- ✓ Marketing personnel who maintain online catalog information
- ✓ Marketing, sales, accounting, and financial managers
- ✓ Senior executives
- ✓ Customers
- ✓ External shippers (e.g., UPS and FedEx)

Looks complete, right?

2 MINUTES · TURN TO YOUR NEIGHBOR

Name 3 stakeholders the charter forgot.

Hint buckets:

- Who keeps the OLD systems running?
- Who touches money or law?
- Which CUSTOMERS can't use a website?
- Who handles things that go wrong?

Hidden stakeholders: the food-delivery app

Pick the food-delivery app you actually use.

Customer-side

- Customer
- Customer support agent
- Anyone without a smartphone

Supply-side

- Restaurant owner & kitchen staff
- Rider / driver
- Grocery pickers

Money & rules

- Payment processor
- Platform investors
- City regulators: labor & food-safety rules

Easy to forget

- App stores (Apple/Google)
- Data/analytics team
- Rider's insurance company

Whose requirements CONFLICT? (customer wants speed · restaurant wants accuracy · rider wants fair time estimates · city wants compliance)

3

The case: Rocky Mountain Outfitters

Strategy → architecture → the pain that justifies CSMS · six ways to gather information

Projects start with strategy, not code

Rocky Mountain Outfitters (RMO) sells outdoor gear through catalogues, phone/mail orders, retail stores, and the web. Customer expectations of modern technology + competitive pressure convinced management it was time to upgrade sales & marketing support. Out of the corporate strategic-planning process came a major project: the **Consolidated Sales & Marketing System (CSMS)**.

Why 'strategic'? An information-systems strategic plan starts from the organization's overall strategic needs, then defines two architectures:

TECHNOLOGY architecture

The set of computing hardware, network hardware & topology, and system software employed by the organization.

RMO reality: existing data center; SCM runs Java on Oracle; phone-order runs Visual Studio on MS SQL.

APPLICATION architecture

The information systems that support the organization: systems, subsystems, and their supporting technology.

RMO reality: SCM, phone/mail order, retail store, customer support, the map on the next two slides.

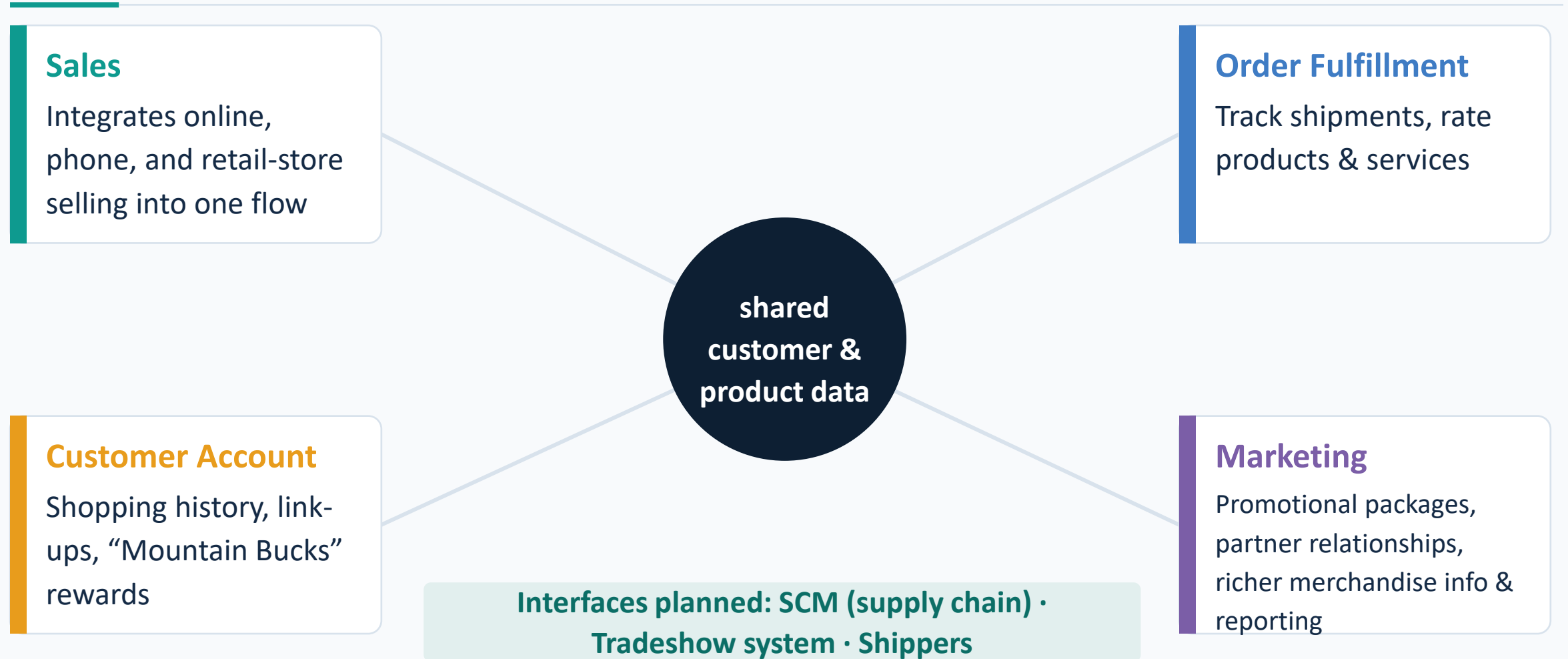
Four systems, four tech stacks, almost zero integration

Supply Chain Mgmt (SCM)	Phone/Mail Order System	Retail Store System	Customer Support (CSS)
• ≈ 5 years old • Java on Oracle • Tradeshow system will interface with it • Healthiest of the four	X • ≈ 12 years old • Visual Studio / MS SQL • Reached capacity, a hard limit • Minimal integration	X • Older packaged solution • Bought, not built • Minimal integration • Store staff are stakeholders too	X • Web-based • Evolved over the years • Minimal integration • Absorbs every failure of the other 3

THE PROBLEM, IN REQUIREMENTS LANGUAGE

No single view of the customer, the same person exists 4 times. Phone orders can't scale past capacity. Every new channel (web, retail, phone) re-implements sales logic. **That's the pain CSMS must fix and every requirement you'll gather traces back to it.**

CSMS: four integrated subsystems, one view of the customer



Six information-gathering techniques

Interviewing

users & other stakeholders, one-on-one or in groups

- ▲ Best: depth, follow-ups, trust
- ▼ Watch: slow; people say ≠ do

Questionnaires

distribute & collect from many people

- ▲ Best: breadth, hundreds of clerks/customers cheaply
- ▼ Watch: low response; no probing

Reviewing documents, inputs & outputs

forms, reports, business rules, screen dumps

- ▲ Best: the AS-IS system & official rules
- ▼ Watch: documents can be outdated

Observing business procedures

watch real work happen, take notes

- ▲ Best: what people ACTUALLY do, incl. workarounds
- ▼ Watch: Hawthorne effect, being watched changes behavior

Researching vendor solutions

demos, references, packaged-software fit

- ▲ Best: buy-vs-build reality check
- ▼ Watch: demos show the happy path

Collecting user comments & suggestions

help-desk tickets, comment cards, support logs

- ▲ Best: real pains, already happening
- ▼ Watch: loudest voices dominate

Analyst's rule: never trust a single technique, triangulate. What the clerk SAYS (interview) + what the forms SHOW (documents) + what you SEE (observation).

Interviewing: ask so the truth comes out

OPEN

“Walk me through what happens when a phone order comes in, start at the ring.”

why: discovers the workflow, surprises, workarounds

CLOSED

“How many returns do you process per day, 5, 50, 500?”

why: pins down quantities for performance & capacity NFRs

PROBING

“You said ‘we usually refund.’ When do you NOT?”

why: business rules hide inside the word 'usually'

FIELD RULES

- Ask for the ACTUAL forms, reports, and screens, artifacts don't lie
- Take notes, then play them back: “So if I understood you correctly...” (member check)
- Interview users separately, then compare stories
- Never lead: “the new system will save you time, right?” poisons the data
- Never promise features, you gather, the team decides

4

From information to models

Text · graphics · math · UML · activity diagrams & swimlanes

From raw information to models

Model: a representation of some important aspect of the system being built. Analysts model BEFORE coding for the same reason architects draw blueprints: mistakes on paper cost ~nothing.

Textual written descriptions

- Use-case narratives
- Business rules with IDs (BR-01...)
- Requirement lists & tables

Graphical diagrams & schematics

- Activity diagrams (today!)
- Use-case diagrams (next lecture)
- Class & sequence diagrams (design phase)

Mathematical formulas, statistics, algorithms

- Capacity: calls/hour per clerk
- Queueing: wait time at holiday peak
- Cost models for buy-vs-build

UML (Unified Modelling Language): the standard graphical modelling notation for information systems. Standard = developers AND business people can read the same picture. Today's tool: the **activity diagram**, it models workflows.

Nine symbols, one workflow

Workflow = the sequence of processing steps that completely handles ONE business transaction or customer request. An activity diagram shows the activities, WHO does each (swimlanes), and the flow between them.



initial node: flow starts

Check stock

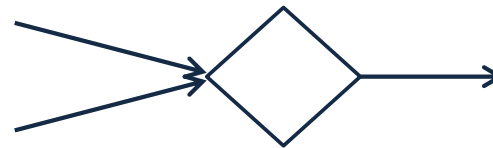
Action: one step of work



Flow: order of steps



Decision: branch on a condition



Merge: branches come back together



fork / join: parallel paths



final node: flow ends

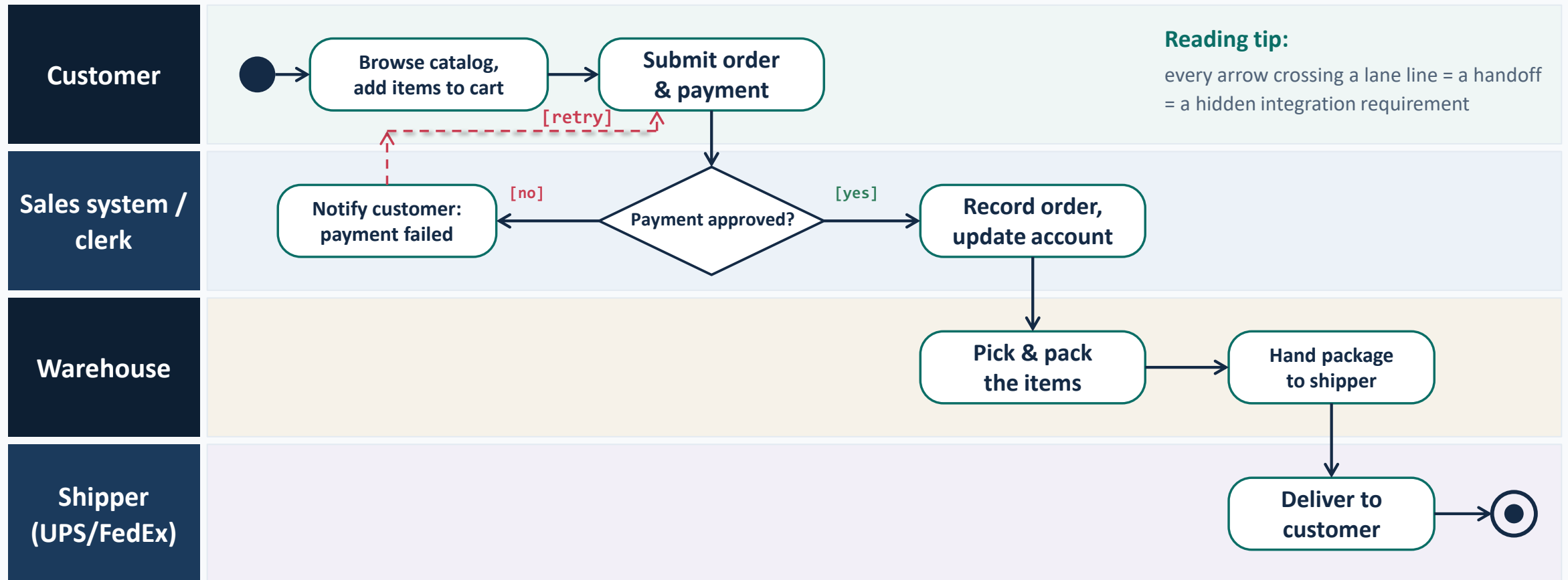


Swimlane: who does each step

[subtotal ≥ \$50] → earn 5%

guard: condition on a flow

RMO: placing a phone/mail order (activity diagram)



Dashed = loop-back (retry). The two lane-crossing handoffs (sales → warehouse, warehouse → shipper) are exactly where the CSMS integration requirements live.

5

From requirements to code

Traceability · business rules in classes · tests that prove requirements · prioritizing

A requirement you can RUN

BR-01 (business rule)

“On every COMPLETED order with subtotal $\geq$ \$50.00, the customer earns Mountain Bucks equal to 5% of the subtotal, rounded to the nearest cent. Orders below \$50 earn nothing.”

1

Write it with an ID

BR-01: unique, citable

2

Code cites the ID

class & method reference BR-01

3

Test cites the ID

test_BR01_... proves it works

Why the ID? When BR-01 changes to 7%, one search finds every place it lives: doc, code, tests, reports.

requirements.py

```
# requirements.py
from decimal import Decimal

REWARD_RATE = Decimal("0.05")      # BR-01: 5%
MIN_SUBTOTAL = Decimal("50.00")    # BR-01: threshold

class Customer:
    def __init__(self, cid, name):
        self.customer_id, self.name = cid, name
        self.mountain_bucks = Decimal("0.00") # BR-02: never < 0

    def earn_rewards(self, subtotal):
        """BR-01: 5% Mountain Bucks if subtotal >= $50."""
        if subtotal < MIN_SUBTOTAL:
            return Decimal("0.00")
        earned = (subtotal * REWARD_RATE).quantize(Decimal("0.01"))
        self.mountain_bucks += earned
        return earned
```

Tests that name the requirement they prove

```
tests.py
# tests.py, each test cites its requirement ID
def test_BR01_earns_five_percent():
    c = Customer("C-1001", "Ali")
    earned = c.earn_rewards(Decimal("120.00"))
    assert earned == Decimal("6.00")          # 5% of 120

def test_BR01_no_reward_below_threshold():
    c = Customer("C-1002", "Aoife")
    assert c.earn_rewards(Decimal("49.99")) == Decimal("0.00")

def test_BR02_balance_never_negative():
    c = Customer("C-1003", "Abigail")
    c.earn_rewards(Decimal("100.00"))          # balance = 5.00
    assert c.redeem_rewards(Decimal("999.00")) is False
    assert c.mountain_bucks == Decimal("5.00") # unchanged!

def test_NFR_PERF_01_under_50ms():           # a measured NFR!
    ...time each call, assert worst < 50 ms...
```

ACTUAL OUTPUT – python3 requirements_demo.py

```
[PASS] BR-01  earns 5% on $120 order
[PASS] BR-01  no reward on $49.99 order
[PASS] BR-02  balance can never go negative
[PASS] NFR-PERF-01  worst call 0.009 ms < 50 ms
```

- Which line proves BR-02? Which proves an NFR?
- If marketing changes 5% → 7%, what do you touch?
- Can a test prove “user-friendly”? What do we do instead?

Prioritize: because everything can't be essential

ESSENTIAL

system is useless without it

RMO: Record & fulfill orders · process payments
· one customer record

IMPORTANT

strongly needed; system works without it briefly

RMO: Mountain Bucks rewards · shipment tracking

NICE TO HAVE

valuable if time & budget remain

RMO: Partner promo packages · advanced merchandise reporting

Industry name for this: **MoSCoW**: Must / Should / Could / Won't (this release). Same idea, four letters.

MoSCoW Prioritization Technique



MUST HAVE

Absolutely essential elements that must be included at any cost.



SHOULD HAVE

Important elements that should only be omitted with careful consideration.



COULD HAVE

Desirable elements that would ideally be included if resources allow.



WON'T HAVE

Elements that are out of scope, unfeasible or actively counterproductive.

@engineeringly

Read the Caption!

Six things to remember on the exam and on the job

- 1 Projects are won or lost before coding. ~⅓ fully succeed; requirements problems are a top root cause.
- 2 FR = what it does · NFR = how well. FURPS+ is your don't-forget-a-category checklist.
- 3 Requirements live in stakeholders. Miss a quadrant, miss a batch of requirements.
- 4 Gather with several techniques and triangulate: say (interview) vs show (documents) vs do (observation).
- 5 Models make workflows discussable, activity diagrams + swimlanes expose WHO, IN WHAT ORDER, and every handoff.
- 6 Every requirement → ID → code → test. Traceability turns promises into proof.

SOURCES & FURTHER READING

Satzinger, Jackson & Burd — Systems Analysis and Design in a Changing World (RMO case; analysis activities; FURPS+). Standish Group CHAOS Report 2020. GAO-14-694 & HHS-OIG (HealthCare.gov, 2014). The Guardian, Sep 2013 (NHS NPfIT ~£10B). DIA baggage: Wrike & Project Insider case studies.